Advanced Programming (OOP) Lab

(PCCCS495)

Project Title: Library Management System

Student Name: Aditya Chatterjee

Roll Number: 38

Instructor Name: Swagatam Basu, Soumadip Biswas, Susovan Jana

Spring 2026

2. Problem Statement

A library requires an efficient system to manage books, members, and transactions. Traditional manual systems often lead to inefficiencies such as difficulty in tracking issued books, maintaining accurate records, and identifying available resources. These issues result in delays, data inconsistency, and reduced user satisfaction.

In manual systems, librarians must record transactions physically, increasing the risk of human error. Searching for a book becomes time-consuming, and updating availability status is not instantaneous. Additionally, maintaining member records manually can lead to duplication and data loss.

To overcome these challenges, a software-based Library Management System is required. The system should automate book management, member registration, issue and return processes, and searching functionality. It should ensure accuracy, improve efficiency, and provide real-time updates.

The objective is to design a modular and scalable system using Object-Oriented Programming principles, ensuring maintainability and future extensibility.

3. System Design

Overview

The system follows a modular Object-Oriented architecture where each real-world entity is represented as a class.

Description of Major Classes

Book Class

- Attributes: id, title, author, isIssued
- Methods:
    - issueBook()
    - returnBook()
    - display()

Member Class

- Attributes: memberId, name
- Methods:
    - display()

Library Class

- Attributes:

- ❖ List<Book> books
- ❖ List<Member> members

- Methods:
  - ❖ addBook()
  - ❖ removeBook()
  - ❖ issueBook()
  - ❖ returnBook()
  - ❖ searchBook()
  - ❖ displayBooks()

Main Class

- ❖ Provides menu-driven interface
- ❖ Takes user input
- ❖ Calls library methods

Package Structure

src/

- ➢ Book.java
- ➢ Member.java
- ➢ Library.java
- ➢ Main.java

Interaction Flow

A. User selects an option

B. Main class receives input

C. Calls Library method

D. Library processes request

E. Result displayed

<u>4. OOP Concepts Demonstrated</u>

4.1 Encapsulation

Where Used:

In all classes (private variables)

Code Snippet:

```
private int id;

private String title;

public int getId() {

    return id;

}
```

4.2 Abstraction

Where Used:

Library methods hide internal logic

Code Snippet:

```
public void issueBook(int bookId) {
    Book b = findBook(bookId);
    if (b != null) b.issueBook();
}
```

Explanation:

User does not know internal implementation.

4.3 Polymorphism

Where Used:

Display methods across classes

Code Snippet:

```
public void display() {
    System.out.println(title);
}
```

Explanation:

Same method name behaves differently for different classes.

4.4 Exception Handling

Where Used:

User input handling

Code Snippet:

```
try {
    choice = sc.nextInt();
} catch (Exception e) {
```

```
    System.out.println("Invalid input");
}
```

Explanation:

Prevents program crash on invalid input.

4.5 Collections Framework

Where Used:

ArrayList

Code Snippet:

```
List<Book> books = new ArrayList<>();
```

Explanation:

Allows dynamic storage of data.

5. Implementation Highlights

Snippet 1: Issue Book Logic

```
if (b == null) {
    System.out.println("Invalid Book ID!");
} else if (b.isIssued()) {
    System.out.println("Already issued!");
} else {
    b.issueBook();
}
```

Justification:

Ensures correct validation before issuing book.

Snippet 2: Search Function

```
for (Book b : books) {
    if (b.getTitle().equalsIgnoreCase(title)) {
        b.display();
    }
}
```

Justification:

Efficient search using loop and string matching.

Snippet 3: Duplicate Prevention

```
if (b.getId() == book.getId()) {
    System.out.println("Duplicate ID!");
```

```
    return;
}
```

Justification:

Ensures data integrity.

Snippet 4: Remove Book

```
books.remove(b);
```

Justification:

Allows dynamic modification of collection.

6. Testing & Error Handling

TEST CASE	INPUT	EXPECTED OUTPUT
Add book	Valid data	Book Added
Issue book	Valid ID	Book Issued
Invalid ID	Wrong data	Error message displayed

Edge Cases

- Duplicate Book ID
- Issuing already issued book
- Returning non-issued book
- Empty book list

Failure Scenarios

- Invalid user input
- Null book reference
- Incorrect ID

Handled using:

- Validation checks
- Exception handling

7. Git Workflow

Commit Strategy

- Minimum 10 commits
- Each commit adds/improves feature
- Incremental development

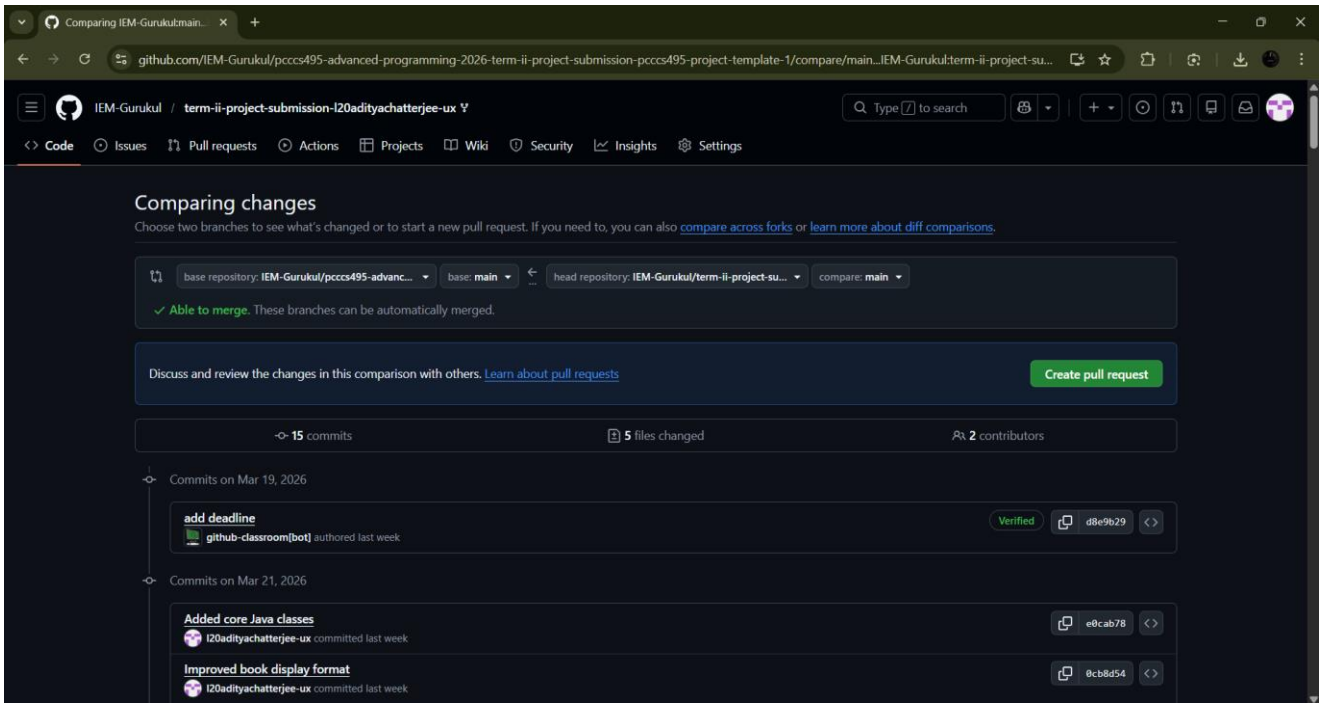
Example Commits

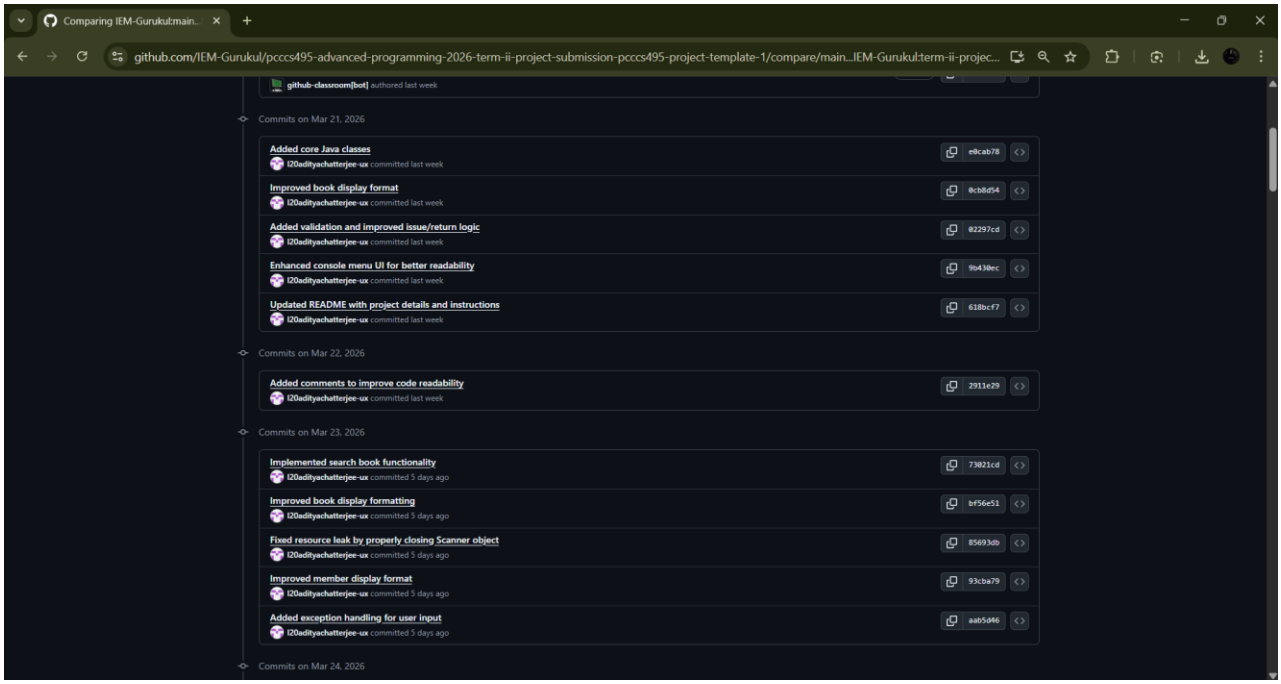
- Added core classes
- Improved UI
- Added validation
- Implemented search feature
- Fixed resource leak

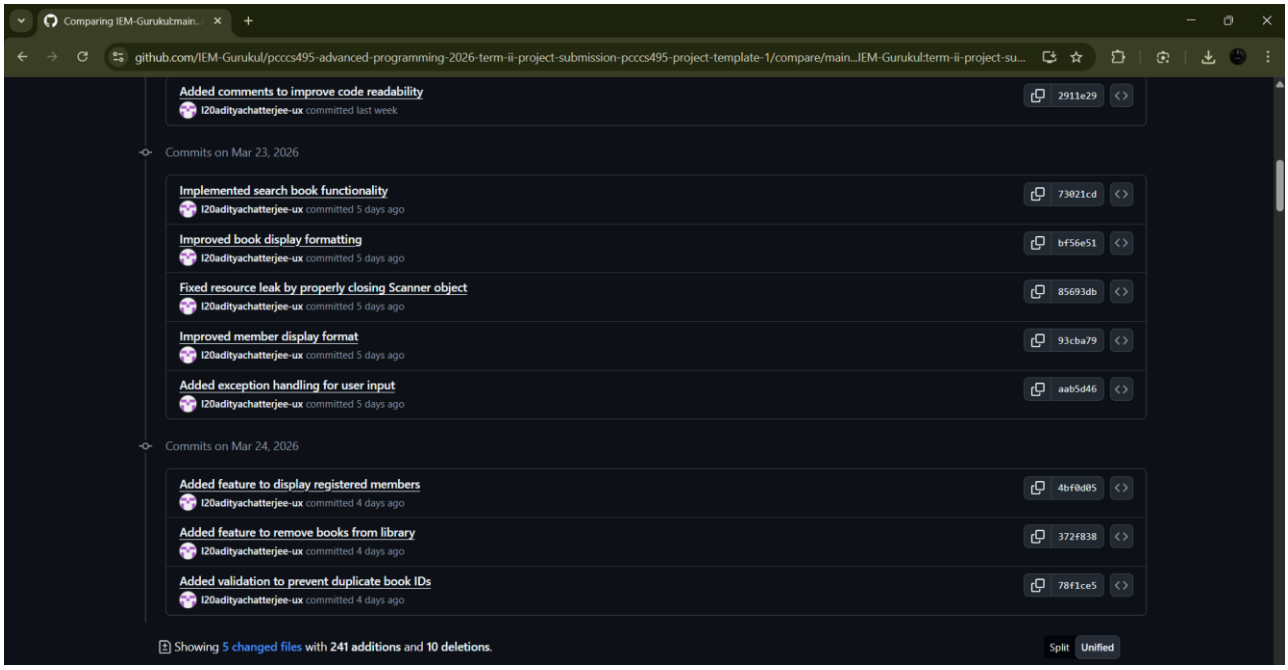
Development Progression

The project was developed step-by-step:

1. Initial structure
2. Core functionality
3. UI improvement
4. Feature addition
5. Error handling
6. Final optimization







8. Conclusion & Future Scope

Conclusion

The Library Management System successfully automates core library operations using Object-Oriented Programming. It improves efficiency, reduces errors, and ensures better data management. The modular design makes the system easy to maintain and extend.

Future Scope

- GUI implementation

- Database integration
- Online access system
- Authentication system
- Mobile application

Appendix:

1.Copy of initial project proposal

Library Management System

Problem Statement:

If a library has a poorly designed software, it can lead to inefficiencies. These include difficulty in tracking issued books, maintaining accurate membership records, and monitoring the availability of book copies. Librarians may face problems such as misplaced records, delays in locating books, and errors while issuing or returning them. These challenges reduce the overall efficiency of library operations and affect the user experience. A well-designed Library Management System can automate these tasks and ensure accurate management of books and member activities. The proposed LMS will help librarians manage books, members, and transactions efficiently using a modular and object-oriented design. It will also allow library members to search for books, issue or return them, and check their availability in real time. By applying OOP principles, the system will be scalable, easy to maintain, and flexible enough to support future extensions such as digital libraries and online reservation services.

Target User

- Librarians
- Library Administrators
- Students / Members

Core Features

- Add, update, and remove books from the library database
- Register and manage library members
- Issue books to members and track due dates
- Return books and update availability status
- Search books by title, author, or ID
- Display available and issued books
- Basic record management and error handling

OOP Concepts to be Used

- Abstraction
- Inheritance
- Polymorphism
- Exception Handling
- Collections

Proposed Architecture Description
The library management system will utilize a modular object-oriented architecture consisting of things like books, users, and library operations that would be represented with different classes. There will be one Library class that contains lots of information related to book collections and members by utilizing data structures like lists or maps. The Book, Member, and Transaction classes will be used to represent the main object types in this library management system. The Library class will provide the service methods to perform actions such as adding or removing books, registering members, issuing books, returning books, and searching for records. The user interface (which will be a console-based user interface) will interact with these classes that perform actions requested by either a librarian or a user. Exception handling would be used to handle errors associated with issuing books that are not available, invalid member number, etc. This modular structure will allow the separation of object responsibilities into coherent abstractions and will make it easier to maintain, enhance or modify in the future.

2. Copy of the review mail received with APPROVED_MINOR(proof attached below)

Dear Aditya Chatterjee,

Project: Library Management System

Decision: APPROVED_MINOR

Score: 9

Strengths:

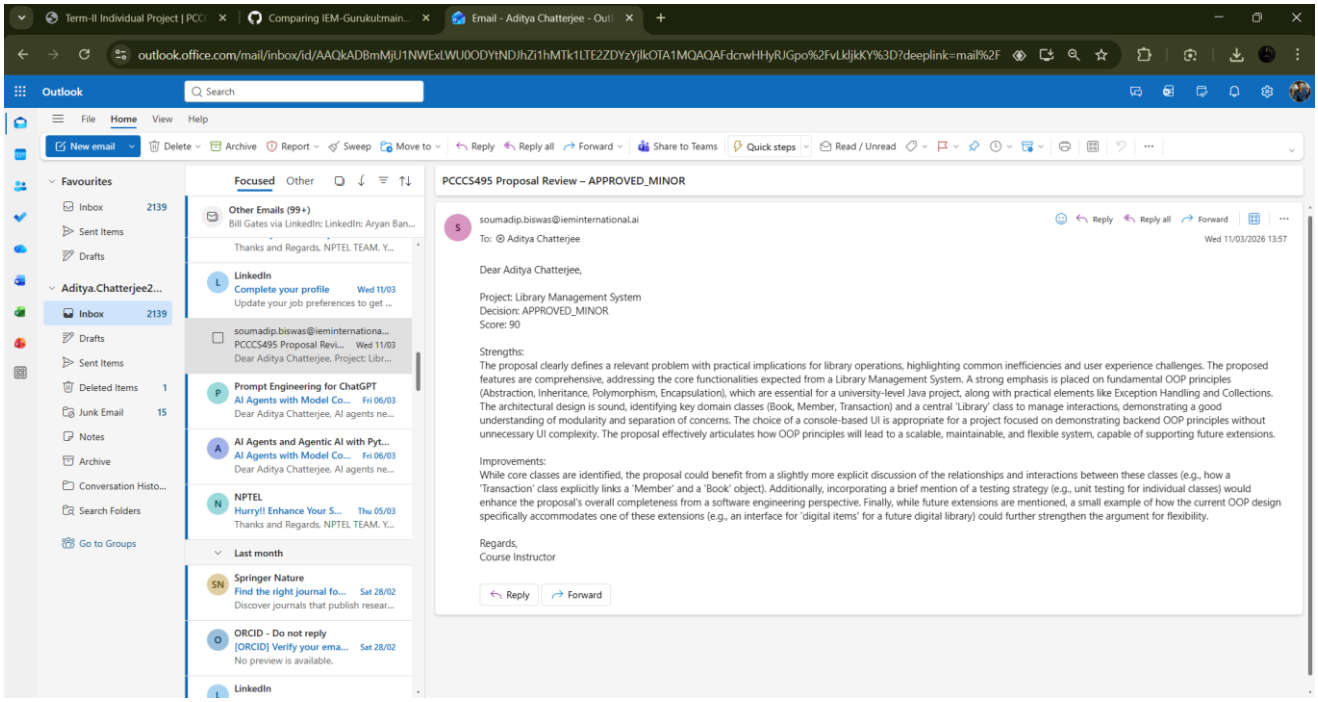
The proposal clearly defines a relevant problem with practical implications for library operations, highlighting common inefficiencies and user experience challenges. The proposed features are comprehensive, addressing the core functionalities expected from a Library Management System. A strong emphasis is placed on fundamental OOP principles (Abstraction, Inheritance, Polymorphism, Encapsulation), which are essential for a university-level Java project, along with practical elements like Exception Handling and Collections. The architectural design is sound, identifying key domain classes (Book, Member, Transaction) and a central 'Library' class to manage interactions, demonstrating a good understanding of modularity and separation of concerns. The choice of a console-based UI is appropriate for a project focused on demonstrating backend OOP principles without unnecessary UI complexity. The proposal effectively articulates how OOP principles will lead to a scalable, maintainable, and flexible system, capable of supporting future extensions.

Improvements:

While core classes are identified, the proposal could benefit from a slightly more explicit discussion of the relationships and interactions between these classes (e.g., how a 'Transaction' class explicitly links a 'Member' and a 'Book' object). Additionally, incorporating a brief mention of a testing strategy (e.g., unit testing for individual classes) would enhance the proposal's overall completeness from a software engineering perspective. Finally, while future extensions are mentioned, a small example of how the current OOP design specifically accommodates one of these extensions (e.g., an interface for 'digital items' for a future digital library) could further strengthen the argument for flexibility.

Regards,

Course Instructor


PCCCS495 Proposal Review – APPROVED_MINOR
soumadip.biswas@ieminternational.ai
To: Aditya Chatterjee
Wed 11/03/2026 13:57
Dear Aditya Chatterjee,
Project: Library Management System
Decision: APPROVED_MINOR
Score: 90
Strengths:
The proposal clearly defines a relevant problem with practical implications for library operations, highlighting common inefficiencies and user experience challenges. The proposed features are comprehensive, addressing the core functionalities expected from a Library Management System. A strong emphasis is placed on fundamental OOP principles (Abstraction, Inheritance, Polymorphism, Encapsulation), which are essential for a university-level Java project, along with practical elements like Exception Handling and Collections. The architectural design is sound, identifying key domain classes (Book, Member, Transaction) and a central 'Library' class to manage interactions, demonstrating a good understanding of modularity and separation of concerns. The choice of a console-based UI is appropriate for a project focused on demonstrating backend OOP principles without unnecessary UI complexity. The proposal effectively articulates how OOP principles will lead to a scalable, maintainable, and flexible system, capable of supporting future extensions.
Improvements:
While core classes are identified, the proposal could benefit from a slightly more explicit discussion of the relationships and interactions between these classes (e.g., how a 'Transaction' class explicitly links a 'Member' and a 'Book' object). Additionally, incorporating a brief mention of a testing strategy (e.g., unit testing for individual classes) would enhance the proposal's overall completeness from a software engineering perspective. Finally, while future extensions are mentioned, a small example of how the current OOP design specifically accommodates one of these extensions (e.g., an interface for 'digital items' for a future digital library) could further strengthen the argument for flexibility.
Regards,
Course Instructor
Reply
Forward